Module: Strings

Phase: Core Python
Previous module: Flow Control
Next module: Lists

Module Outcome:

- Slice and clean names, emails, and UPI IDs without breaking Unicode
- Format a receipt line (`f"₹{amount}"`) instead of glueing commas by hand
- Spot immutable-string traps before they silently drop an edit

What is a string?

A **string** (`str`) is a sequence of characters — letters, digits, symbols, spaces — used for names, emails, addresses, JSON text, and HTML.

Bank UPI Employee Order Hospital

Where used?
- **SecureBank** / UPI — mask account/phone digits, parse UPI IDs
- College apps — roll numbers, email normalize for login
- **TechMart** — product titles, URL slugs, invoice lines
- **AcmeHR** — offer-letter templates, employee name formatting
- **MedCare** — patient phone masking, ticket subject cleanup

Why do we need strings?

Almost every app stores **text**: student name, email, phone, product title, error message. Before lists and databases, you must clean and slice text (today's lab kit: email/phone cleaner).

Without string skills	With string skills
Messy `"  Ravi@Gmail.COM "` breaks login	`strip()` + `lower()` normalizes input
Show full phone on screen	Mask as `******3210` for privacy
Manual parsing of `"a,b,c"`	`split` / `join` for CSV-lite lines

```
name = "Ananya Sharma"
course = 'Python Full Stack'
bio = """Joined July 2026 batch.
Based in Bengaluru."""
```

Single and double quotes behave the same. **Triple quotes** allow multiple lines.

Python has **no separate `char` type** — even `'A'` is a string of length 1.

Indexing — what / why, then syntax

Definition: Each character has a position number starting at `0`.
Purpose: Extract initials, domain from email, last 4 digits of a phone.

Indices start at **0** for the first character.

```
  P  y  t  h  o  n
  0  1  2  3  4  5   ← positive index
 -6 -5 -4 -3 -2 -1   ← negative index (from end)
```

```
lang = "Python"
print(lang[0])    # P
print(lang[-1])   # n (last char)
```

Slicing — what / why, then syntax

Definition: Take a contiguous piece of the string with `string[start:stop:step]`.
Purpose: Pull username from email, reverse a string for drills, skip characters.

Syntax: `string[start:stop:step]`

- **start** included, **stop** excluded (same rule as `range`)
- Omit start → from beginning; omit stop → to end

```
email = "ravi.kumar@company.com"
print(email[0:4])      # ravi
print(email[5:10])     # kumar
print(email[-4:])      # .com
print(email[::2])      # every 2nd char — rai.kmr@cmpyom
```

Immutability — what / why

Definition: You cannot change a character inside an existing string object.
Purpose: Predict bugs like `name[0] = "A"` (fails); use reassignment / new strings instead (`name = name.upper()`).

Immutable means: once created, the string object cannot change. Methods return **new** strings.

```
city = "mumbai"
city.upper()     # returns "MUMBAI" — city still "mumbai"
print(city)      # mumbai

city = city.upper()   # rebinding variable — OK
# city[0] = "M"       # TypeError — cannot assign to index
```

Why it matters: passing strings between functions is safe — no accidental in-place edits.

Common methods (job daily drivers)

Method	Purpose	Example
`upper()` / `lower()`	Case change	Normalize email
`strip()`	Remove surrounding whitespace	Clean `input()`

Method	Purpose	Example
`split(sep)`	Break into list	Parse `"a,b,c"`
`join(iterable)`	Build string from pieces	Build CSV line
`replace(old, new)`	Substitute text	Mask digits
`find(sub)`	Index or -1	Search domain
`startswith` / `endswith`	Prefix/suffix test	`.pdf` files
`count(sub)`	Occurrences	Count spaces

Formatting strings

f-strings (preferred):

```
emp = "E101"
name = "Divya"
print(f"Employee {emp}: {name}")
print(f"{'Name':<20}{'ID':>6}")
print(f"{'Divya':<20}{'E101':>6}")
```

Important Concepts and Definitions

Term	Definition
String (str)	Immutable sequence of Unicode characters
Index	Integer position of character (0-based)
Slice	Substring specified by start:stop:step
Immutable	Object state cannot change after creation
Escape sequence	`\n` newline, `\t` tab, `\\` backslash, `\'` quote
Method	Function tied to object: `s.lower()`
Whitespace	Spaces, tabs, newlines — often stripped from input
Delimiter	Character separating parts: comma, pipe, `@`
Format specifier	f-string `{value:width.precision}` control
Slug	URL-friendly lowercase hyphenated identifier

Step-by-Step Notes (With Annotated Code)

Step 1: Creating strings and escapes

```
# strings_basics.py

single = 'KSR Trainings'
```

```
double = "Python Full Stack"
multi = """Line 1: Student Management
Line 2: Employee Portal
Line 3: Expense Tracker"""

# Escape sequences
path = "C:\\Users\\Ravi\\projects"   # backslash in Windows path
quote = "She said, \"Let's code.\""  # escaped double quotes
newline = "First line\nSecond line"

print(multi)
print(newline)
```

Note:
- Wrong: Comparing or adding strings to numbers (`"42" + 1`); using `is` for string *content* (`a is "hi"`).
- Prefer: Convert (`int("42") + 1`); use `==` for value compare.

Print a multi-line KSR Student Portal banner using a triple-quoted string.

```
msg = """KSR Student Portal
1. Enter your roll number
2. View marks and attendance"""
print(msg)
```

Store a Windows path with backslashes and print it.

```
path = "C:\\Users\\Priya\\projects"
print(path)
```

Step 2: Indexing and slicing — employee email

```
email = "priya.nair@datavizon.in"

username = email[:email.find("@")]       # priya.nair
domain = email[email.find("@") + 1:]    # datavizon.in
first_letter = email[0].upper()

print("Username:", username)
print("Domain:", domain)
print("Initial:", first_letter)
```

Alternative with split (cleaner):

```
username, domain = email.split("@")
```

Note:
- Wrong: Indexing past the end (`s[100]` on a short string); expecting `s[0:3]` to be 4 characters; confusing negative-step slices like `s[::-1]`.
- Prefer: Check `len(s)` first; remember stop is exclusive (3 chars); draw an index diagram for reverse slices.

Split ravi.kumar@ksr.in at @ and print username and domain.

```
email = "ravi.kumar@ksr.in"
at = email.find("@")
print("username:", email[:at])
print("domain:", email[at + 1 :])
```

Expected Output:

```
username: ravi.kumar
domain: ksr.in
```

Print first and last characters and the slice name[1:4] for Priya.

```
name = "Priya"
print(name[0], name[-1])
print(name[1:4])  # riy
```

Expected Output:

```
P a
riy
```

Step 3: Immutability demo

```
status = "pending"
print(id(status))

status = status.replace("pending", "approved")   # new string object
print(status)          # approved
print(id(status))      # different id than before
```

Note:

- Wrong: Calling `s.upper()` and expecting `s` to change (`print(s)` still old).
- Prefer: Assign the result: `s = s.upper()`.

Replace pending with paid in a new string and show the original is unchanged.

```
msg = "pending"
msg2 = msg.replace("pending", "paid")
print(msg)
print(msg2)
```

Expected Output:

```
pending
paid
```

Remember: Strings are **immutable** — `replace` / `upper` return a *new* string; rebind if you want to keep the change (`s = s.upper()`).

Rebuild Ravi with X as the first letter (comment out the invalid index assignment).

```
s = "Ravi"
# s[0] = "X"  # TypeError: strings are immutable — use replace or rebuild
s2 = "X" + s[1:]
print(s, s2)
```

Step 4: Case and strip — login normalization

```
# Case-insensitive login compare (lab pattern)

stored_email = "admin@company.com"
entered = input("Email: ").strip().lower()

if entered == stored_email:
    print("Login OK")
else:
    print("Invalid email")
```

Annotation:

- `strip()` removes accidental spaces from copy-paste
- `lower()` avoids case mismatch

Note:
- Wrong: Skipping `.strip()` on input — `" admin "` fails a login compare.
- Prefer: `.strip()` (and usually `.lower()`) on user-entered text.

Normalize a spaced, uppercase email with strip and lower, then compare and print it.

```
raw = "  PRIYA@KSR.IN  "
normalized = raw.strip().lower()
print(normalized == "priya@ksr.in")
print(normalized)
```

Expected Output:

```
True
priya@ksr.in
```

Common Error: Comparing login email without `.strip().lower()` — `" Admin@KSR.IN "` fails even when the account exists.

Strip and title-case a messy name string.

```
name = "  ravi kumar  "
print(name.strip().title())
```

Step 5: split and join — task list from CSV line

```
line = "Design UI, Build API, Write tests, Deploy"
tasks = line.split(", ")          # list of 4 tasks
print(tasks)

# join back with pipe separator
pipe_format = " | ".join(tasks)
print(pipe_format)
```

Student marks line parse:

```
record = "2026042|Priya|78|85|72"
parts = record.split("|")
roll, name, py, sql, html = parts
print(f"Roll {roll}: {name} — Python {py}")
```

Note:
- Wrong: Splitting with the wrong delimiter (`"a,b,c".split(", ")` vs `split(",")`).
- Prefer: Match the exact separator in the data.

Split comma-separated menu items and join them with | separators.

```
items = "Chai,Samosa,Dosa".split(",")
print(" | ".join(items))
```

Parse a pipe-separated marks record and print roll, name, and Python marks.

```
roll, name, python, sql, html = "2026003|Ravi|88|72|91".split("|")
print(roll, name, python)
```

Step 6: replace and count — phone masking

```
phone = "9876543210"
masked = phone[:2] + "******" + phone[-2:]
print(masked)   # 98******10

text = "error: failed error: retry"
print(text.count("error"))        # 2
print(text.replace("error", "warn"))  # new string
```

Mask a 10-digit phone number showing only the first two and last two digits.

```
phone = "9876543210"
masked = phone[:2] + "*" * 6 + phone[-2:]
print(masked)  # 98******10
```

Expected Output:

```
98******10
```

Count fail occurrences in text and replace fail with warn.

```
text = "fail retry fail ok"
print(text.count("fail"))
print(text.replace("fail", "warn"))
```

Expected Output:

```
2
warn retry warn ok
```

Step 7: startswith, endswith, in — file and email checks

```
filename = "report_q1.pdf"

if filename.lower().endswith(".pdf"):
    print("PDF document")

url = "https://portal.company.com/login"
if url.startswith("https://"):
    print("Secure URL")

email = "user@example.com"
if "@" in email and "." in email.split("@")[-1]:
    print("Email looks structurally OK (lite check)")
```

Check whether invoice_july.pdf ends with .pdf case-insensitively.

```
filename = "invoice_july.pdf"
print(filename.lower().endswith(".pdf"))
```

Run a lite email validation: require @ and a dot in the domain.

```
email = "priya@ksr.in"
if "@" in email:
    domain = email.split("@", 1)[1]
    if "." in domain:
        print("OK")
    else:
        print("Invalid domain")
else:
    print("Missing @")
```

Step 8: f-string formatting — offer letter snippet

```
name = "Karthik Reddy"
role = "Python Developer Intern"
ctc = 450000

letter = f"""
Dear {name},

We are pleased to offer you the role of {role}.
CTC: ₹{ctc:,} per annum.
```

```
Regards,
HR Team
"""
print(letter)
```

Print a short offer letter with candidate name, role, and formatted CTC.

```
name = "Priya"
role = "Intern"
ctc = 480000
print("Offer Letter")
print(f"Candidate: {name}")
print(f"Role: {role}")
print(f"CTC: ₹{ctc:,}")
```

Print aligned Name and Role columns for two employees.

```
print(f"{'Name':<10}{'Role':<10}")
print(f"{'Ravi':<10}{'Backend':<10}")
print(f"{'Meera':<10}{'QA':<10}")
```

Step 9: Job-oriented — generate URL slug

```
# slug from course title for LMS URL

title = "  Python Full Stack – July 2026 Batch!!!  "
slug = title.strip().lower()
slug = slug.replace(" ", "-")
# remove simplistic "invalid" chars for demo
for ch in "!":
    slug = slug.replace(ch, "")

print(slug)   # python-full-stack---july-2026-batch
# Production: use regex module — taught later
```

Convert a course title to a URL slug with strip, lower, and space-to-hyphen.

```
title = " Python Training Notes "
slug = title.strip().lower().replace(" ", "-")
print(slug)
```

Clean a noisy course title to lowercase hyphenated form without exclamation marks.

```
raw = "Full Stack!!! Python"
clean = raw.strip().lower().replace(" ", "-").replace("!", "")
print(clean)
```

Step 10: Loop + strings — count vowels

```
name = input("Enter name: ")
vowels = "aeiouAEIOU"
count = 0
for ch in name:
```

```
    if ch in vowels:
        count += 1
print(f"Vowels in '{name}': {count}")
```

Count vowels in Ananya Sharma.

```
name = "Ananya Sharma"
vowels = "aeiouAEIOU"
count = sum(1 for ch in name if ch in vowels)
print(count)
```

Expected Output:

```
5
```

Count digit characters in a messy phone string.

```
phone = "98a76b543210"
digits = sum(1 for ch in phone if ch.isdigit())
print(digits)
```

Practice Exercises

Basic

1. Given `s = "Full Stack Python"`, print first char, last char, length.

Model answer:

```
s = "Full Stack Python"
print(s[0])      # F
print(s[-1])     # n
print(len(s))    # 17
```

2. Slice to extract `"Stack"` from the string above.

Model answer:

```
s = "Full Stack Python"
print(s[5:10])  # Stack
```

3. Convert `" hello world "` to `"HELLO WORLD"` using methods only.

Model answer:

```
raw = "  hello world  "
clean = raw.strip().upper()
print(clean)  # HELLO WORLD
```

4. Split `"apple,banana,mango"` by comma; print each fruit on new line with for.

Model answer:

```
fruits = "apple,banana,mango".split(",")
for fruit in fruits:
    print(fruit)
```

Intermediate

1. **Employee portal:** Read email; print username and domain using split.

Model answer:

```
email = input("Email: ").strip()
username, domain = email.split("@", 1)
print(f"Username: {username}")
print(f"Domain: {domain}")
```

2. Mask Aadhaar-style number: show only last 4 digits (`XXXX-XXXX-1234`).

Model answer:

```
aadhaar = input("12-digit Aadhaar: ").strip().replace(" ", "").replace("-", "")
last4 = aadhaar[-4:]
print(f"XXXX-XXXX-{last4}")
```

3. Check password length >= 8 and contains at least one digit (loop + `in`).

Model answer:

```
password = input("Password: ")
has_digit = False
for ch in password:
    if ch in "0123456789":
        has_digit = True
        break
if len(password) >= 8 and has_digit:
    print("Password OK")
else:
    print("Need length >= 8 and at least one digit")
```

4. Reverse a string using slicing `[::-1]` and using a loop (two ways).

Model answer:

```
s = "Namaste"
print(s[::-1])  # slicing

reversed_s = ""
for ch in s:
    reversed_s = ch + reversed_s
print(reversed_s)  # loop
```

Interview-Oriented

1. Are strings mutable in Python? What happens with `s[0] = 'x'`?

Model answer: Strings are **immutable**. You cannot change a character in place. `s[0] = 'x'` raises `TypeError`. To "change" text, build a new string (slice + concat, `replace`, etc.) and rebind the name.

2. Difference between `" ".join(list)` and `+` concatenation in a loop? (Performance: join faster.)

Model answer: Building with `result += piece` in a loop creates many temporary strings (quadratic cost in the naive case). `" ".join(parts)` builds the result once from a list — preferred for joining many pieces. Use `+` for a few fixed parts; use `join` for lists of unknown length.

3. Explain `s[2:8:2]` on `"abcdefghij"`.

Model answer: `s = "abcdefghij"` → indices 0…9. Slice `s[2:8:2]` starts at index 2 (`c`), stops before 8, step 2 → `c`, `e`, `g` → `"ceg"`.

4. How check palindrome without reversing? (Two-pointer idea — or compare to `[::-1]`.)

Model answer: Simple interview answer: `s == s[::-1]` (after normalizing case/spaces if asked). Two-pointer idea without building a reverse copy: left at 0, right at `len(s)-1`; while left < right, compare `s[left]` and `s[right]`; if mismatch, not palindrome; else move inward.

`python s = "level" left = 0 right = len(s) - 1 is_pal = True while left < right: if s[left] != s[right]: is_pal = False break left += 1 right -= 1 print(is_pal)`

Mini-Project (Assignment): AcmeHR / Student Text Utilities CLI

Title: Menu-driven text cleaner for registration fields
Lab kit (build here): 15_Lab_Assets/09_string_parser/

What to build

Menu-driven program (`text_utils.py`):

1. Normalize name (title case + strip)
2. Validate email (lite: one `@`, non-empty parts, domain has `.`)
3. Mask mobile number
4. Parse marks string `roll,name,m1,m2,m3` → formatted report
5. Generate slug from course name
6. Exit

Use functions optional — single file with while menu is fine.

Sample marks parse input:

```
2026001,Ravi Kumar,88,76,91
```

Sample output:

```
Roll: 2026001
Name: Ravi Kumar
Average: 85.00
```

Stretch

- Mask Aadhaar-style input showing only last 4 digits.
- Reject emails with spaces after strip.

Done when

- [ ] Lab kit string-parser ticket cleans email/phone as expected
- [ ] Menu options 1–5 work and exit on `0`
- [ ] Marks parse prints roll, name, and average

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why might HR store employee names as strings even if name looks numeric?
2. Is `"admin@company.com" == "Admin@company.com"` True? Should login be case-sensitive?
3. When is splitting better than slicing for parsing?
4. WhatsApp message vs Python string — both text; what extra does Python string give?
5. GDPR/privacy: why mask phone numbers in logs?

Q1. Is Python string mutable?
A: No. str is immutable; methods like replace return new strings; assignment rebinds variable name.

Q2. Difference between `find` and `index`?
A: Both search substring; `find` returns -1 if missing; `index` raises ValueError.

Q3. What does `s[::-1]` do?
A: Reverses string with slice step -1.

Q4. Why use `join` instead of + in loop?
A: join is efficient for many concatenations; + creates many intermediate objects.

Q5. How validate email in production vs beginner lab?
A: Lab: structural checks with in/split; production: regex or validator library plus verification mail.
